

Windows PowerShell

D:\Study\3 CCE\Term 8\Networks\Labs\Lab 1>tcp_server.py

[TCP] Server listening on 127.0.0.1:65432

[TCP] New connection from ('127.0.0.1', 65478)

[TCP] Active connections: 1

[TCP] Received from ('127.0.0.1', 65478): Aman

[TCP] Sending to ('127.0.0.1', 65478): nma

[TCP] Received from ('127.0.0.1', 65478): Aabc

[TCP] Sending to ('127.0.0.1', 65478): cba

[TCP] Received from ('127.0.0.1', 65478): Cname

[TCP] Sending to ('127.0.0.1', 65478): aemn

[TCP] Received from ('127.0.0.1', 65478): Dwan

[TCP] Sending to ('127.0.0.1', 65478): WAN

[TCP] Received from ('127.0.0.1', 65478): DlowCase

[TCP] Sending to ('127.0.0.1', 65478): LOWCASE

PS D:\Study\3 CCE\Term 8\Networks\Labs\Lab 1> .\client.py tcp

[TCP Client] Connecting to 127.0.0.1:65432...

[TCP Client] Connected! Type 'quit' to exit.

You: Aman

Server: nma

You: Aabc

Server: cba

You: Cname

Server: aemn

You: Dwan

Server: WAN

You: DlowCase

Server: LOWCASE

You: |

Windows PowerShell

Microsoft Windows [Version 10.0.26200.7840]
(c) Microsoft Corporation. All rights reserved.

D:\Study\3 CCE\Term 8\Networks\Labs\Lab 1>udp_server.py
[UDP] Server listening on 127.0.0.1:65433
[UDP] Received from ('127.0.0.1', 57106): Agarab
[UDP] Sending to ('127.0.0.1', 57106): rgbaa
[UDP] Received from ('127.0.0.1', 57106): Aabcdef
[UDP] Sending to ('127.0.0.1', 57106): fedcba
[UDP] Received from ('127.0.0.1', 57106): CsomeThing
[UDP] Sending to ('127.0.0.1', 57106): Teghimnos
[UDP] Received from ('127.0.0.1', 57106): Cabcde
[UDP] Sending to ('127.0.0.1', 57106): abcde
[UDP] Received from ('127.0.0.1', 57106): DlowerCase
[UDP] Sending to ('127.0.0.1', 57106): LOWERCASE
|

PS D:\Study\3 CCE\Term 8\Networks\Labs\Lab 1> .\client.py udp
[UDP Client] Sending to 127.0.0.1:65433
[UDP Client] Ready! Type 'quit' to exit.

You: Agarab
Server: rgbaa

You: Aabcdef
Server: fedcba

You: CsomeThing
Server: Teghimnos

You: Cabcde
Server: abcde

You: DlowerCase
Server: LOWERCASE

You: |

000000

```
udp.port == 65433
```

No.	Time	Source	Destination	Protocol	Length	Info
6	3.867432700	127.0.0.1	127.0.0.1	UDP	36	65433 → 57106 Len=4
11	6.722954800	127.0.0.1	127.0.0.1	UDP	32	57106 → 65433 Len=0
12	6.723288500	127.0.0.1	127.0.0.1	UDP	32	65433 → 57106 Len=0
17	10.935383500	127.0.0.1	127.0.0.1	UDP	37	57106 → 65433 Len=5
18	10.935596900	127.0.0.1	127.0.0.1	UDP	36	65433 → 57106 Len=4
47	42.231110700	127.0.0.1	127.0.0.1	UDP	37	57106 → 65433 Len=5
48	42.231335900	127.0.0.1	127.0.0.1	UDP	36	65433 → 57106 Len=4
49	42.679701600	127.0.0.1	127.0.0.1	UDP	37	57106 → 65433 Len=5
50	42.679911900	127.0.0.1	127.0.0.1	UDP	36	65433 → 57106 Len=4
51	43.131002800	127.0.0.1	127.0.0.1	UDP	37	57106 → 65433 Len=5
52	43.131215300	127.0.0.1	127.0.0.1	UDP	36	65433 → 57106 Len=4
53	43.837727200	127.0.0.1	127.0.0.1	UDP	37	57106 → 65433 Len=5
54	43.838107100	127.0.0.1	127.0.0.1	UDP	36	65433 → 57106 Len=4
98	85.774863100	127.0.0.1	127.0.0.1	UDP	37	57106 → 65433 Len=5
99	85.775116900	127.0.0.1	127.0.0.1	UDP	36	65433 → 57106 Len=4
136	98.381390100	127.0.0.1	127.0.0.1	UDP	38	57106 → 65433 Len=6
137	98.381628300	127.0.0.1	127.0.0.1	UDP	37	65433 → 57106 Len=5
138	99.138947400	127.0.0.1	127.0.0.1	UDP	38	57106 → 65433 Len=6
139	99.139238300	127.0.0.1	127.0.0.1	UDP	37	65433 → 57106 Len=5
144	101.827730100	127.0.0.1	127.0.0.1	UDP	36	57106 → 65433 Len=4
145	101.827981200	127.0.0.1	127.0.0.1	UDP	35	65433 → 57106 Len=3

```

Frame 136: Packet, 38 bytes on wire (304 bits), 38 bytes captured (304 bits) on interface \Device\NPF_{...}
Null/Loopback
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
User Datagram Protocol, Src Port: 57106, Dst Port: 65433
Data (6 bytes)

```

```
0000  02 00 00 00 45 00 00 22  86 8f 00 00 40 11 f6 39  ....E-" .....@.9
0010  7f 00 00 01 7f 00 00 01  df 12 ff 99 00 0e 11 eb  ....
0020  41 6d 61 6e 6e 5c                Amann\
```

```
1 import socket
2 import sys
3
4 TCP_HOST = '127.0.0.1'
5 TCP_PORT = 65432
6 UDP_HOST = '127.0.0.1'
7 UDP_PORT = 65433
8
9
10 def tcp_client():
11     print(f"[TCP Client] Connecting to {TCP_HOST}:{TCP_PORT}...")
12     with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as sock:
13         sock.connect((TCP_HOST, TCP_PORT))
14         print("[TCP Client] Connected! Type 'quit' to exit.\n")
15
16         while True:
17             message = input("You: ")
18             if message.lower() == 'quit':
19                 break
20
21             sock.sendall(message.encode('utf-8'))
22             response = sock.recv(1024).decode('utf-8')
23             print(f"Server: {response}\n")
24
25     print("[TCP Client] Disconnected.")
26
27
28 def udp_client():
29     print(f"[UDP Client] Sending to {UDP_HOST}:{UDP_PORT}")
30     print("[UDP Client] Ready! Type 'quit' to exit.\n")
31
32     with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as sock:
33         while True:
34             message = input("You: ")
35             if message.lower() == 'quit':
36                 break
37
38             sock.sendto(message.encode('utf-8'), (UDP_HOST, UDP_PORT))
39             response, _ = sock.recvfrom(1024)
```

client.py > ...

```
27
28 def udp_client():
29     print(f"[UDP Client] Sending to {UDP_HOST}:{UDP_PORT}")
30     print("[UDP Client] Ready! Type 'quit' to exit.\n")
31
32     with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as sock:
33         while True:
34             message = input("You: ")
35             if message.lower() == 'quit':
36                 break
37
38             sock.sendto(message.encode('utf-8'), (UDP_HOST, UDP_PORT))
39             response, _ = sock.recvfrom(1024)
40             print(f"Server: {response.decode('utf-8')}\n")
41
42     print("[UDP Client] Done.")
43
44
45 def main():
46     if len(sys.argv) != 2 or sys.argv[1].lower() not in ('tcp', 'udp'):
47         print("Usage: python client.py <tcp|udp>")
48         sys.exit(1)
49
50     mode = sys.argv[1].lower()
51     if mode == 'tcp':
52         tcp_client()
53     else:
54         udp_client()
55
56
57 if __name__ == '__main__':
58     main()
59
```



```
1  import socket
2  import threading
3
4  HOST = '127.0.0.1'
5  PORT = 65432
6
7
8  def process_message(message: str) -> str:
9      if len(message) == 0:
10         return message
11
12     command = message[0]
13     data = message[1:]
14
15     if command == 'A':
16         # Sort characters in descending order
17         return ''.join(sorted(data, reverse=True))
18     elif command == 'C':
19         # Sort characters in ascending order
20         return ''.join(sorted(data))
21     elif command == 'D':
22         # Convert all letters to uppercase
23         return data.upper()
24     else:
25         # Return the exact same message
26         return message
27
28
29 def handle_client(conn: socket.socket, addr: tuple):
30     print(f"[TCP] New connection from {addr}")
31     with conn:
32         while True:
33             try:
34                 data = conn.recv(1024)
35                 if not data:
36                     # Client disconnected
37                     break
38                 message = data.decode('utf-8').strip()
39                 print(f"[TCP] Received from {addr}: {message}")
```

```
29 def handle_client(conn: socket.socket, addr: tuple):
30     # Client disconnected
31     break
32     message = data.decode('utf-8').strip()
33     print(f"[TCP] Received from {addr}: {message}")
34
35     response = process_message(message)
36     print(f"[TCP] Sending to {addr}: {response}")
37     conn.sendall(response.encode('utf-8'))
38 except ConnectionResetError:
39     break
40 except Exception as e:
41     print(f"[TCP] Error with {addr}: {e}")
42     break
43 print(f"[TCP] Connection closed: {addr}")
44
45
46 def main():
47     server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
48     server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
49     server.bind((HOST, PORT))
50     server.listen()
51     print(f"[TCP] Server listening on {HOST}:{PORT}")
52
53     try:
54         while True:
55             conn, addr = server.accept()
56             # Create a new thread for each client
57             thread = threading.Thread(target=handle_client, args=(conn, addr), daemon=True)
58             thread.start()
59             print(f"[TCP] Active connections: {threading.active_count() - 1}")
60     except KeyboardInterrupt:
61         print("\n[TCP] Server shutting down.")
62     finally:
63         server.close()
64
65
66 if __name__ == '__main__':
67     main()
```



```
7 def process_message(message: str) -> str:
18     # Sort characters in ascending order
19     return ''.join(sorted(data))
20 elif command == 'D':
21     # Convert all letters to uppercase
22     return data.upper()
23 else:
24     # Return the exact same message
25     return message
26
27
28 def main():
29     server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
30     server.bind((HOST, PORT))
31     print(f"[UDP] Server listening on {HOST}:{PORT}")
32
33     try:
34         while True:
35             data, client_addr = server.recvfrom(1024)
36             message = data.decode('utf-8').strip()
37             print(f"[UDP] Received from {client_addr}: {message}")
38
39             response = process_message(message)
40             print(f"[UDP] Sending to {client_addr}: {response}")
41             server.sendto(response.encode('utf-8'), client_addr)
42     except KeyboardInterrupt:
43         print("\n[UDP] Server shutting down.")
44     finally:
45         server.close()
46
47
48 if __name__ == '__main__':
49     main()
50
```

```
1  import socket
2
3  HOST = '127.0.0.1'
4  PORT = 65433
5
6
7  def process_message(message: str) -> str:
8      if len(message) == 0:
9          return message
10
11     command = message[0]
12     data = message[1:]
13
14     if command == 'A':
15         # Sort characters in descending order
16         return ''.join(sorted(data, reverse=True))
17     elif command == 'C':
18         # Sort characters in ascending order
19         return ''.join(sorted(data))
20     elif command == 'D':
21         # Convert all letters to uppercase
22         return data.upper()
23     else:
24         # Return the exact same message
25         return message
26
27
28  def main():
29      server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
30      server.bind((HOST, PORT))
31      print(f"[UDP] Server listening on {HOST}:{PORT}")
32
33      try:
34          while True:
35              data, client_addr = server.recvfrom(1024)
36              message = data.decode('utf-8').strip()
37              print(f"[UDP] Received from {client_addr}: {message}")
38
39              response = process_message(message)
```